
ELEC9451 / ELEC4951



UNSW
THE UNIVERSITY OF NEW SOUTH WALES

School of Electrical Engineering
and Telecommunications

THESIS/PROJECT PART A
TECHNICAL REPORT COVER-SHEET

Study the marking guide and the requirements given overleaf. Complete the details required on this form and include it at the front of your report (first page).

Student name: Jordan Mohindra

Student ID: z5418455

Thesis/Project title: Comparison and Implementation of Data Compression Methods for SoC-based Radar Systems

Supervisor: A/Prof Beena Ahmed

Assessor: Dr. Mostafa Shahin

I declare that this assessment item is my own work, except where acknowledged, and has never been submitted for academic credit elsewhere, and acknowledge that the assessor of this item may, for the purpose of assessing this item,

- (i) reproduce this assessment item and provide a copy to another member of the University, and/or*
- (ii) communicate a copy of this assessment item to a plagiarism checking service (which may then retain a copy of the assessment item on its database for the purpose of future plagiarism checking).*

I certify that I have read and understood the University Rules about Student Academic Misconduct.

Signature: jmohindra

Date: 22/04/2026

**SCHOOL OF ELECTRICAL ENGINEERING
AND TELECOMMUNICATIONS**

Comparison and Implementation of Data Compression Methods for SoC-based Radar Systems

by

Jordan Mohindra

Student ID: z5418455

Thesis submitted as a requirement for the degree
Bachelor of Engineering (Electrical Engineering)

Submitted: April 22, 2026
Supervisor: Beena Ahmed

Abstract

With the introduction of high-resolution MIMO radars into automotive ADAS systems, there have been ever-growing bandwidth requirements. Transmitting large amounts of raw ADC data to a central System-on-Chip without overwhelming both the chip and communication lines is therefore a significant challenge.

This thesis addresses this problem by comparing and implementing available data compression techniques to mitigate these bandwidth requirements. This research builds upon a baseline study of existing methods and extends the literature by addressing some of the key issues identified in prior work.

A simulated environment is constructed to benchmark various emerging and existing data compression techniques. The most viable algorithm is then implemented on hardware, where various metrics are measured to ultimately assess its viability.

Acknowledgements

I would like to thank Dr. Beena Ahmed for the guidance she has provided thus far.

Abbreviations

ACC Adaptive Cruise Control

ADAS Advanced Driver Assistance Systems

ADC Analog-to-Digital Converter

AEB Automatic Emergency Braking

ALU Arithmetic Logic Unit

ASIC Application-Specific Integrated Circuit

BAQ Block Adaptive Quantization

BFP Block Floating Point

BRAM Block Random Access Memory

BSD Blind Spot Detection

CR Compression Ratio

CSI Camera Serial Interface

DCT Discrete Cosine Transform

DMA Direct Memory Access

DRHE Doppler Redundancy Hybrid Encoding

DSP Digital Signal Processing

ECU Electronic Control Unit

EGE Exponential-Golomb Encoding

EVM Evaluation Module

FMCW Frequency-Modulated Continuous Wave

FN False Negative

FP False Positive

FPGA Field-Programmable Gate Array

FPU Floating-Point Unit

HDL Hardware Description Language

HLS High-Level Synthesis

HPC High-Performance Computer

LCA Lane Change Assist

MCU Microcontroller Unit

MIMO Multiple-Input Multiple-Output

MMIC Monolithic Microwave Integrated Circuit

MPSoC Multi-Processor System-on-Chip

RADAR Radio Detection And Ranging

RF Radio Frequency

RX Receive / Receiver

SAR Synthetic Aperture Radar

SDV Software-Defined Vehicle

SerDes Serialiser/Deserialiser

SNR Signal-to-Noise Ratio

SoC System-on-Chip

SPT Signal Processing Toolbox

TI Texas Instruments

Contents

Acknowledgements	1
Abbreviations	2
Contents	4
1 Introduction	6
1.1 Motivation	6
1.2 Objective	8
2 Background	10
2.1 Automotive Radar	10
2.1.1 Introduction of Automotive Radar	10
2.1.2 The AWR2243 Radar Sensor	12
2.1.3 MMWCAS-RF-EVM	12
2.2 Data Compression for Automotive Applications	13
2.2.1 Introduction to Compression Algorithms	13
2.2.2 The Data Compression Process	14
2.2.3 Compression Algorithm Metrics	14
2.2.4 Automotive Compression Algorithm Approaches	16
2.2.5 Neural Network-Based Approaches	16
2.3 Radar Data Compression in Other Fields	17
2.4 Automotive System Architectures	17
2.4.1 Evolution of Automotive Architectures	17
2.4.2 Implications for Automotive Radar Data Management	18
2.5 State of the Art Compression Algorithms	19

2.5.1	Hardware-Accelerated Deterministic Compression (NXP CPX)	19
2.5.2	Block Adaptive Quantization (BAQ)	19
2.5.3	Doppler Redundancy Hybrid Encoding (DRHE)	20
2.5.4	Rate-Adaptive Spectral Compression (AdaRadar)	20
2.6	Field-Programmable Gate Arrays (FPGAs)	21
2.6.1	Strategic Advantages for Radar DSP	22
2.6.2	Deployment and Production Considerations	22
2.7	Hardware Number Representations	23
2.7.1	Fixed-Point Representation	23
2.7.2	Floating-Point Representation	24
2.7.3	Quantization Error in Fixed-Point Systems	24
3	Thesis Outline	26
3.1	Baseline Study: Kiem's Thesis	26
3.1.1	Overview and Objectives	26
3.1.2	Results and Algorithm Evaluation	26
3.1.3	Limitations and Future Work	27
3.2	Proposed Thesis Plan	28
3.2.1	Goal 1: Mitigating Quantization Error	28
3.2.2	Goal 2: Evaluation of Novel Compression Algorithms	28
3.2.3	Goal 3: End-to-End Hardware Simulation on the EVM	29
3.3	Project Timeline	29
3.4	Challenges and Risks	30
3.4.1	FPGA Implementation and Timing Closure	30
3.4.2	Simulating Proprietary MCU Architectures	30
4	Conclusion	31
	Bibliography	32

Chapter 1

Introduction

The automotive industry is currently undergoing a rapid transformation, especially with the pursuit of autonomous vehicles. The main driver of this transformation is the reliance on various sensor technologies that enable the vehicle to interpret its surrounding environment. Amongst these sensor technologies is radar, which is critical mainly due to its ability to work under various adverse conditions, such as rain, fog, snow, and darkness; conditions that other sensors, such as cameras and lidar, would not function in [1][2].

Radar has long been integral to Advanced Driver Assistance Systems (ADAS), initially being used for straightforward tasks such as distance warning and collision avoidance. However, as radar technology has evolved, its applications have expanded rapidly, including adaptive cruise control (ACC), automatic emergency braking (AEB), blind spot detection (BSD), and lane change assist (LCA) [3]. More recently, the trajectory of radar has shifted toward supporting levels of autonomy, including SAE Levels 2+ through 4, which demand high-resolution environmental mapping, object classification, and multi-target tracking [4]. All these advances in radar applications have substantially increased the resolution and complexity requirements.

1.1 Motivation

The motivation behind this research stems from this increase in resolution. As automotive systems architecture has evolved from domain-based to centralized design, the demands on data transmission infrastructure have intensified [5][6]. In a centralized architecture, raw sensor data from multiple radars distributed around the vehicle must be sent to a central processing unit.

Since transmitting data in its raw form allows maximum flexibility for downstream processing, it places a high demand on the vehicle's communication bandwidth [7].

To better illustrate this, consider the architecture seen in Figure 1.1. This architecture is known as Serializer/Deserializer (SerDes) [8][9], which uses high-speed serial data lines to transport data from sensors to a central processor. In this example, various radar modules are placed around the car, with two radar modules aggregated at the front. Each of these radar modules uses 4 receive antennas that transmit at 600 Mbit/s. This data is transmitted from the radar module over 4 parallel data lines using Camera Serial Interface (CSI-2) to a serializer unit, which takes the 4 600 Mbit/s data lines (or in the case of the front module, 8 data lines) and serializes them into a 2.5 Gbit/s data line (5 Gbit/s for the front). Then, at the central System-on-Chip (SoC), a corresponding deserializer reconstructs the parallel data lines. While SerDes architectures are capable of handling these extremely high data rates, they carry the significant disadvantage of requiring expensive shielded cabling to maintain signal integrity, and require the SoC and deserializer to have substantial processing capacity and memory bandwidth to ingest the incoming data [10][11].

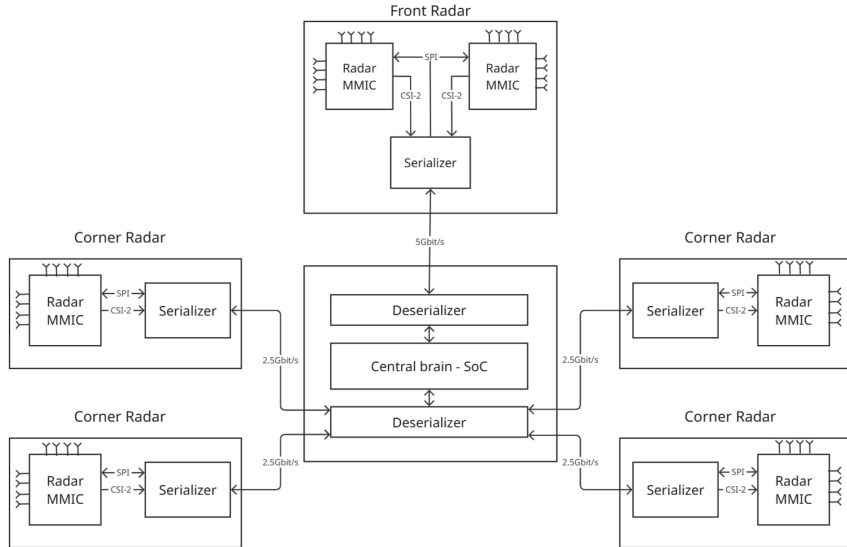


Figure 1.1: Centralized architecture with SerDes [12].

An alternative approach (and the one that motivates this thesis) is to apply data compression directly at the sensor edge before transmission to the SoC, as illustrated in Figure 1.2. By compressing the data at the source, we are able to substantially reduce the required transmission bandwidth. In this example, compressing the 2.5 Gbit/s data stream by a factor of 2.5 or greater

would reduce the throughput to 1 Gbit/s or below, enabling the use of Gigabit Ethernet [13]. Gigabit Ethernet offers a cost-effective and well-standardized interconnect solution, and its adoption would yield reductions in cabling cost and system complexity compared to the SerDes solution

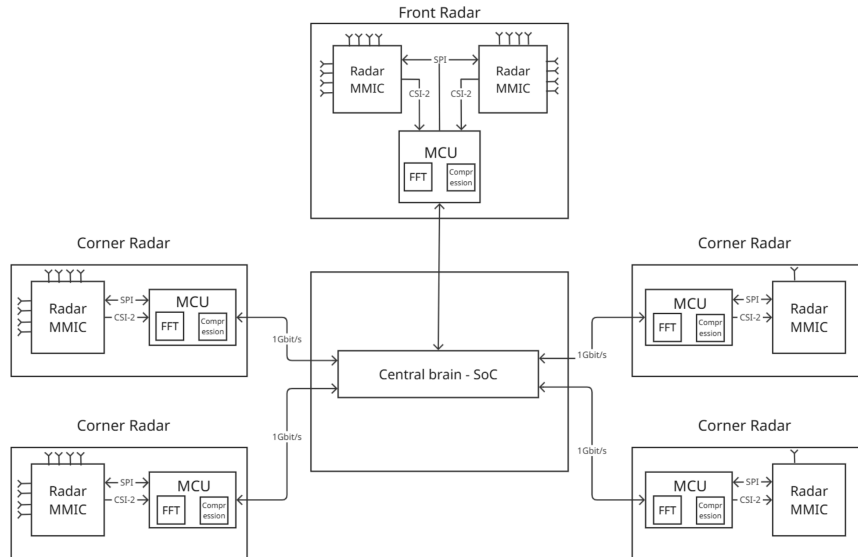


Figure 1.2: Centralized architecture with applied compression[12].

1.2 Objective

The primary objective of this thesis is to extend and improve the radar data compression framework established by Lukas Kiem [12], addressing the critical gaps identified in his work and broadening the evaluation to include more recent compression approaches.

Specifically, this thesis pursues the following objectives:

1. Kiem's implementation required that radar data be converted to a 16-bit fixed-point format prior to compression. This linear quantization step was identified as the dominant source of information loss in the processing chain, as it crushes weak radar signals, which are critical for detecting small or distant objects such as pedestrians, into the noise floor. This thesis investigates non-linear quantization strategies [14].
2. Kiem's FPGA implementation was tested exclusively using simulated dummy data, leaving its performance in real-world scenarios unvalidated. This thesis uses the Texas Instru-

ments AWR2243 cascaded MIMO radar sensor to capture raw, real-world data, enabling more accurate analysis of compression algorithms performance.

3. This thesis expands the list of compression algorithms evaluated, including both newer and older methods that promise the same, if not better, performance metrics than those investigated by Kiem.

The ultimate goal of this thesis is to advance the state of the art in automotive data compression by implementing a solution that achieves high compression ratios whilst meeting the strict requirements of low latency.

Chapter 2

Background

This chapter provides the fundamental knowledge needed to interpret and evaluate the data compression methods discussed in this thesis. This means understanding the operating principles of automotive radar, outlining the compression DSP pipeline, and introducing the hardware platform needed to actually implement the algorithm.

2.1 Automotive Radar

2.1.1 Introduction of Automotive Radar

Modern automotive radar systems predominantly rely on Frequency-Modulated Continuous Wave (FMCW) technology. Unlike traditional pulsed radars, FMCW radars continuously transmit a radio frequency (RF) signal whose frequency increases linearly over a short duration. This frequency sweep is known as a “chirp”, as seen in Figure 2.1(a).

The operation begins with the chirp being transmitted through the transmit (TX) antennas. When this signal encounters an object in the environment, a portion of the energy is reflected back to the receiver (RX) antennas. Because the transmitted frequency is constantly changing, the received echo is a delayed and attenuated version of the transmission, meaning there is a frequency difference (Δf) between what is currently being transmitted and what is being received at any given instant due to the time delay (Δt).

A mixer component combines the transmitted and received signals to produce a low-frequency “beat” or Intermediate Frequency (IF) signal. This IF signal is then passed through a low-pass

filter and digitized by an Analog-to-Digital Converter (ADC). The frequency of this digitized IF signal is directly proportional to the distance of the reflecting object.

As seen in Figure 2.1(b), getting meaningful data from the signal requires a structured DSP pipeline in which to transform the initial 1D range bins into a 3D radar data cube [15]

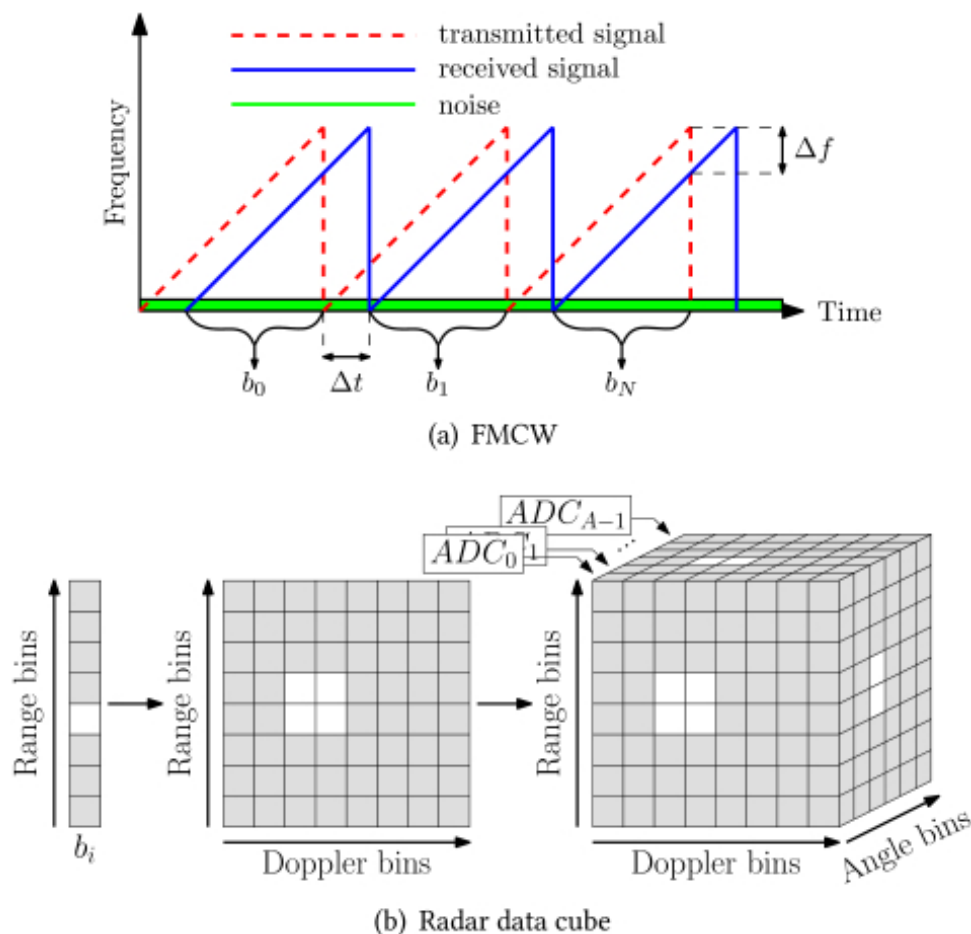


Figure 2.1: (a) Transmitted and received chirp signals over time, and (b) the formation of the 3D Radar Data Cube [15].

This pipeline consists of 3 Fast Fourier Transforms (FFTs):

1. Range FFT (1D FFT): The first FFT is applied to a single chirp. Peaks in the frequency domain correspond to the range (distance) of a detected object.
2. Doppler FFT (2D FFT): Taking a second FFT across the chirps at each range bin, the radar resolves phase shifts caused by the Doppler effect, which allows for the determination of relative velocity.
3. Angle FFT (3D FFT): Using Multiple-Input Multiple-Output (MIMO) antenna arrays, a

third FFT is performed across the spatial domain (the different receive channels). The phase differences between physically separated antennas are used to calculate the Angle of Arrival (AoA).

2.1.2 The AWR2243 Radar Sensor

The AWR2243 is a Texas Instruments FMCW transceiver chip that is designed for automotive applications. It features three TX and four RX channels.

Crucially for this research, the AWR2243 acts as an RF front-end. It handles transmission and reception, mixing, and ADC conversion, but does not perform any DSP pipeline operations (FFTs). Instead, it streams the raw, uncompressed digitized ADC data out via a high-speed MIPI CSI-2 interface. This architecture makes the AWR2243 the ideal platform for evaluating raw data compression algorithms, as it exposes the exact data payload that would cause transmission bottlenecks in a centralized vehicle architecture.

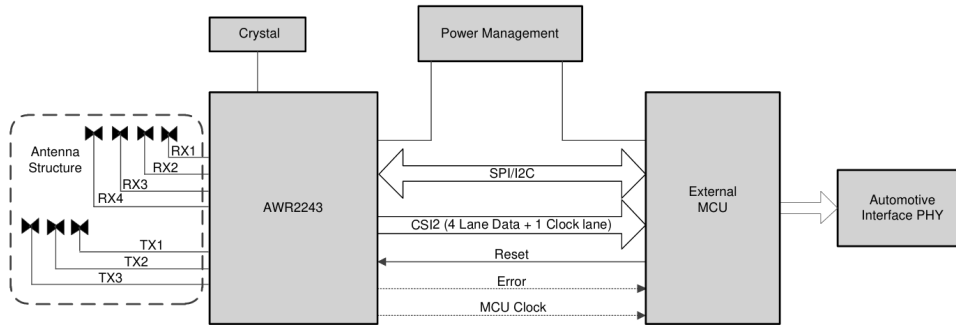


Figure 2.2: AWR2243 Typical Application Diagram [16].

2.1.3 MMWCAS-RF-EVM

To achieve the high angular resolution required for advanced autonomous driving, a single AWR2243 chip with its 3 TX and 4 RX antennas is often insufficient. The MMWCAS-RF-EVM is an evaluation module that addresses this limitation by integrating four AWR2243 transceivers onto a single printed circuit board.

By cascading these four chips, the module creates a synchronized MIMO array comprising 12 transmit and 16 receive antennas. This cascaded setup synthesizes an array of 192 virtual antennas, providing great spatial resolution but simultaneously generating an immense volume of raw data.

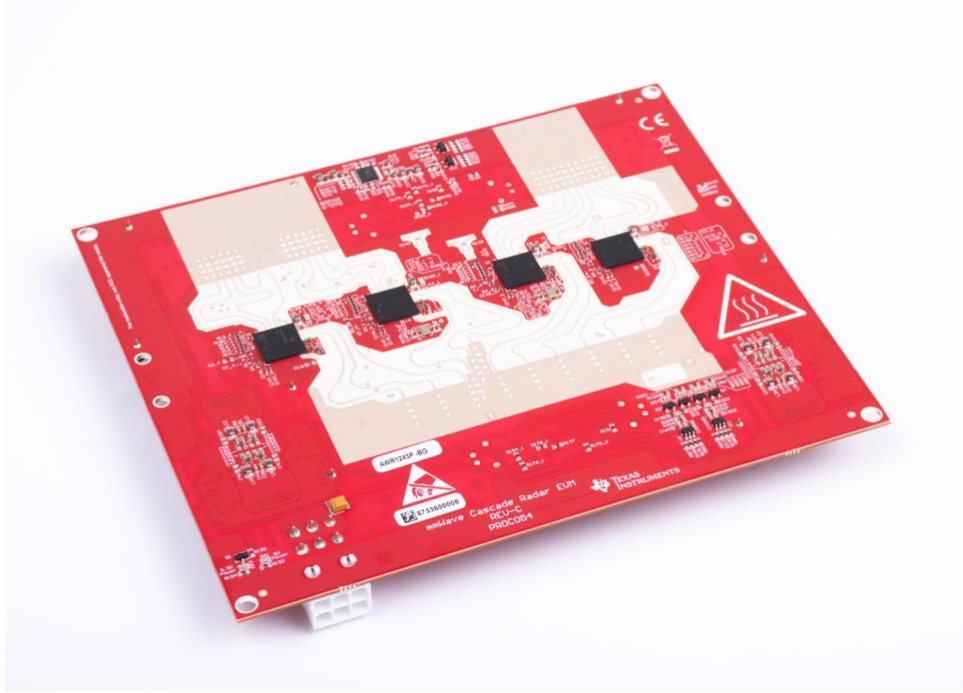


Figure 2.3: Image of MMWCAS-RF-EVM [16].

2.2 Data Compression for Automotive Applications

As MIMO radar scales quadratically with the number of antennas, the size of the resulting radar data cube is very large. A single high-definition radar frame from a cascaded MIMO setup can easily exceed 100 Megabytes. At typical frame rates, this results in a large data stream of multiple Gigabits per second, which overwhelms the communication network and the SoC's processing and memory. To address this issue, we can implement data compression directly on the sensor edge

2.2.1 Introduction to Compression Algorithms

Data compression involves encoding information using fewer bits than what was originally used. Data compression falls into two main categories: lossless and lossy [17].

Lossless compression reduces data size whilst perfectly preserving all original information, meaning the decompressed data fully matches the original input. However, lossless techniques are typically restricted to lower compression ratios.

Lossy compression can achieve much higher data compression ratios by permanently discarding information deemed "less critical". Whilst this results in fewer bits being used, the removed

information is lost. In automotive applications, the choice of what is deemed less critical is vital, as the loss of environmental features can have disastrous consequences. Ultimately, selecting a compression algorithm would require a trade-off between compression ratio and data loss.

2.2.2 The Data Compression Process

The workflow for a data compression algorithm is illustrated in Figure 2.4. The original data is passed through a compression block, which reduces it to a smaller payload. This compressed payload is then either stored in memory or, in the automotive context, sent to the SoC. When it reaches its destination, it is then passed back through a decompression block in order to reconstruct the original signal (as best as possible).

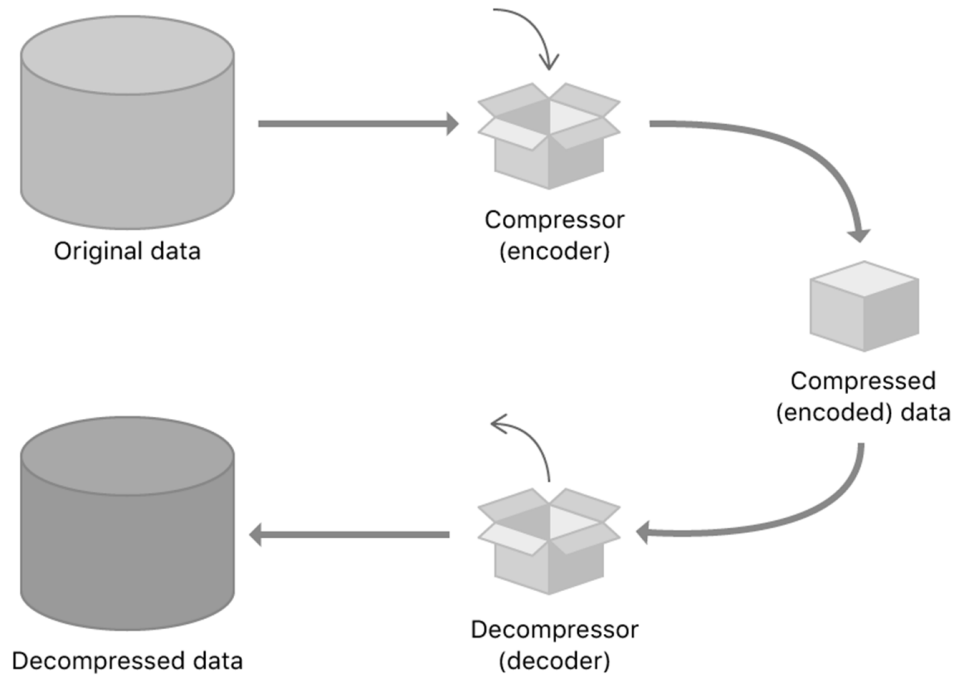


Figure 2.4: Basic Compression and Decompression Cycle [18].

2.2.3 Compression Algorithm Metrics

Since automotive radar is used in critical systems such as Autonomous Emergency Braking (AEB), data compression algorithms cannot blindly remove information. Hence, we must assess the trade-off between the compression efficiency and the decompressed data fidelity. To evaluate this, we can quantifiably assess the performance metrics of these algorithms

Compression Ratio (CR)

The compression ratio is the ratio of the uncompressed data size to the compressed data size:

$$CR = \frac{N_{\text{input bits}}}{N_{\text{output bits}}} \quad (2.1)$$

For example, reducing a 32-bit number to 8 bits gives a compression ratio of 4. Whilst a higher compression ratio is the ultimate goal in our context, it typically results in greater information loss (in lossy algorithms).

Signal-to-Noise Ratio (SNR) and Noise Floor

If the compression algorithm is very aggressive, it introduces noise that raises the noise floor of the decompressed data. This directly affects the Signal to Noise ratio (SNR), which is defined as:

$$SNR_{\text{dB}} = 10 \log_{10} \left(\frac{P_{\text{signal}}}{P_{\text{noise}}} \right) = S_{\text{dB}} - N_{\text{dB}} \quad (2.2)$$

where S_{dB} is the RMS signal power and N_{dB} is the RMS noise floor, both measured in decibels. A high SNR is important for ensuring signal quality; if the SNR is too low, the weak reflections from smaller objects can disappear in the noise.

Detection Accuracy (False Positives and False Negatives)

The detection accuracy is measured by using False Positives (FP) and False Negatives (FN). If the compression algorithm removes too much relevant data, the SoC might not be able to detect a real target, resulting in False Negatives.

This is evaluated using the False Negative Rate (Miss Rate):

$$FNR = \frac{FN}{TP + FN} \quad (2.3)$$

Conversely, if the compression introduces mathematical artifacts or ridges into the range-Doppler map, the system might detect a ‘ghost’ target that does not actually exist, known as a False Positive. This is evaluated using the False Discovery Rate:

$$FDR = \frac{FP}{FP + TP} \quad (2.4)$$

Both of these detection failures are highly dangerous in an autonomous driving environment and must remain as close to zero as possible.

Real-Time Latency

The compression latency (L) is defined as the time difference between receiving the raw frame data (t_{in}) and successfully outputting the compressed data (t_{out}):

$$L = t_{\text{out}} - t_{\text{in}} \quad (2.5)$$

A compression algorithm cannot take hundreds of milliseconds to process a frame. It must operate with minimal latency and massive throughput to ensure true real-time performance at the sensor edge.

2.2.4 Automotive Compression Algorithm Approaches

The current state of the art in radar data compression is split into two paradigms: deterministic time-domain algorithms and spectral/neural algorithms.

Deterministic and Time-Domain Approaches

Because automotive safety systems require low latency, semiconductor manufacturers typically rely on lightweight, deterministic time-domain algorithms. Prominent examples include Texas Instruments' Exponential-Golomb Encoding (EGE) [19], NXP Semiconductors' proprietary CP4 and CP8 formats [20], and Doppler Redundancy Hybrid Encoding (DRHE) [21].

The primary advantage of these methods is their latency; hardware implementations of algorithms such as DRHE can achieve latency as low as 0.08 milliseconds [12]. However, these algorithms generally hit a hard compression ceiling of approximately 3x to 4x.

2.2.5 Neural Network-Based Approaches

Recent advancements have introduced Neural Network (NN) architectures to compress radar data by deploying an encoder at the sensor edge, and a decoder at the SoC [2, 22]. While these models achieve very high compression ratios, deploying them on FPGAs requires significant resources.

Modern frameworks like AdaRadar [23] address this computational burden by avoiding a full neural encoder at the edge. Instead, AdaRadar employs a simple spectral pruning process at the sensor, keeping both the edge compression and central decompression extremely lightweight.

The computationally intensive neural network runs entirely on the SoC, enabling this type of compression in an automotive context.

2.3 Radar Data Compression in Other Fields

While data compression is utilized across various fields, existing methods are generally unsuitable for the automotive sector as they require low latency. Traditional radar compression can often be performed after scanning, and thus does not fall under this strict latency requirement. Automotive radar algorithms must execute with millisecond latency and minimal computational overhead, which precludes the use of complex compression schemes used in other radar applications.

2.4 Automotive System Architectures

2.4.1 Evolution of Automotive Architectures

Automotive system architectures are undergoing a shift with the emergence of the Software-Defined Vehicle (SDV), in which functionality is primarily managed via software rather than hardware. To manage the growing demands of autonomy and connectivity, as seen in automotive radar, SDV architectures have evolved through four distinct stages [24]:

Distributed (Modular) Architecture (SDV 0.0)

In early automotive electronics, features were implemented as standalone systems. Each new vehicle function required a dedicated Electronic Control Unit (ECU), leading to networks with over 100 separate ECUs. This resulted in a very high wiring harness weight and little adaptability to change.

Domain-Based Architecture (SDV 1.0)

To reduce the number of distributed ECUs, the industry grouped them into logical functional domains (e.g., powertrain, chassis, ADAS), each governed by a central domain controller. While this improved internal organization, it still required extensive cross-vehicle wiring because sensors had to connect to their respective domain controllers regardless of their physical

location.

Zonal Architecture (SDV 2.0)

Sensors and actuators are grouped strictly by their physical location in the vehicle (e.g., front-left zone, rear-right zone) rather than their function. Zonal gateways get data from local sensors and route it over high-speed communication lines to a central High-Performance Computer (HPC).

Vehicle-Centralized Architecture (SDV 3.0)

The most recent SDV, where all logic is consolidated into a SoC.

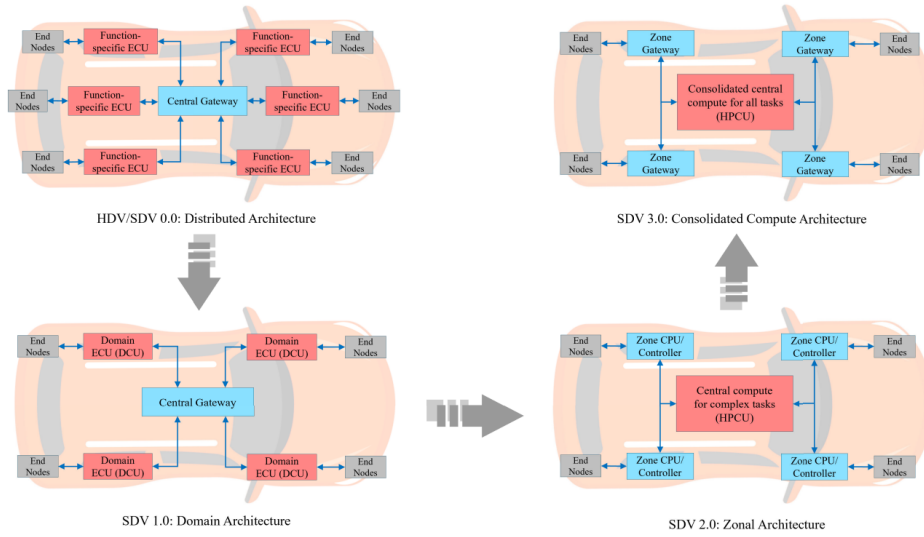


Figure 2.5: The four evolutionary stages of automotive architectures [24]

2.4.2 Implications for Automotive Radar Data Management

In this thesis, the focus is on Zonal Architecture (SDV 2.0), rather than the fully Vehicle-Centralized SDV 3.0 approach. By using the SDV 2.0 (Zonal) architecture, we can compress the data directly at the radar module, significantly reducing the data payload before it enters the network

2.5 State of the Art Compression Algorithms

To address the specific constraints of automotive radar, several compression algorithms have been developed. These algorithms prioritize latency and low computational complexity, with minimal information loss (i.e., low false positives/negatives and SNR). This section outlines the high-level principles of the current industry standards and the baseline algorithms evaluated in this thesis.

2.5.1 Hardware-Accelerated Deterministic Compression (NXP CPX)

NXP has a series of radar processors, such as the S32R45 and the S32R27x, which utilize a hardware accelerator unit called the Signal Processing Toolbox (SPT) to handle the large data volumes [20]. This toolbox is designed to perform various functions, such as FFTs and windowing, with high efficiency compared to other DSPs. To achieve the required low latency, the toolbox uses Programmable Direct Memory Access (PDMA) units, which transfer data from the SPT's RAM to the system's larger SRAM. PDMA applies the compression algorithm, which typically uses Block Floating Point logic. For example, the CP4D format compresses four operands by maintaining eight 7-bit mantissas that share a single 4-bit exponent. These algorithms are designed to be "near lossless" in radar applications, often introducing less than 0.25 dB of signal loss at standard compression ratios.

Compression Mode	Number of Channels / Number of Complex operands (24, 16bit)	Compression Ratio [bits_opram/ bits_ram]	Compression Ratio [bits_ram / bits_ram]
CP4, CP4D	4	3	4
CP4FMTA, CP4DFMTA	4	6	8
CP4FMTB, CP4DFMTB	4	2.82	3.8
CP6	6	4.5	6
CP8FMTA	8	6	8
CP8FMTB	8	2.91	3.9
CP16FMTB	16	2.95	3.9

Figure 2.6: Technical specifications and ratios for the various NXP compression formats [20]

2.5.2 Block Adaptive Quantization (BAQ)

Originally developed for Synthetic Aperture Radar (SAR) [25], Block Adaptive Quantization (BAQ) principles can be adapted to automotive applications. Like NXP, BAQ divides the radar

data into blocks. However, BAQ estimates the statistical properties of the signal, such as its variance and average magnitude.

Based on this estimate, the algorithm adjusts its quantization step size. The blocks containing weak signals (from small objects) are quantized with smaller step sizes, with the stronger blocks using larger step sizes. This allows for a higher dynamic range, at the cost of slightly increased computational complexity.

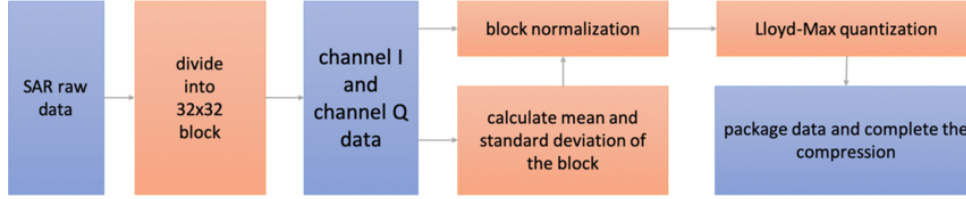


Figure 2.7: Flowchart of BAQ algorithm compressing process [26]

2.5.3 Doppler Redundancy Hybrid Encoding (DRHE)

Doppler Redundancy Hybrid Encoding (DRHE) leverages redundancy in FMCW radar frames for compression. A radar frame consists of identical chirps transmitted in rapid succession. Because the physical scene changes negligibly between consecutive chirps, the received ADC data exhibits extremely high correlation in the slow-time (Doppler) domain [21].

Instead of quantizing the raw data directly, DRHE calculates the difference (delta) between identical samples across consecutive chirps. Because the scene is mostly static relative to the high sampling rate, these differences are overwhelmingly small and cluster heavily around zero.

The DRHE algorithm then applies a lossless variable-length entropy encoder (such as Golomb-Rice encoding) to these deltas. High-probability values near zero are assigned very short bit lengths, achieving significant data reduction. The main disadvantage of this algorithm lies in how to efficiently encode rare, large outliers (e.g., from fast-moving targets or noise spikes) for which standard lossless encoding becomes highly inefficient.

2.5.4 Rate-Adaptive Spectral Compression (AdaRadar)

While traditional algorithms operate in the spatial-temporal domain, recent advancements have introduced dynamic compression. An example is AdaRadar [23], which uses an adaptive feedback loop to adjust its compression ratio.

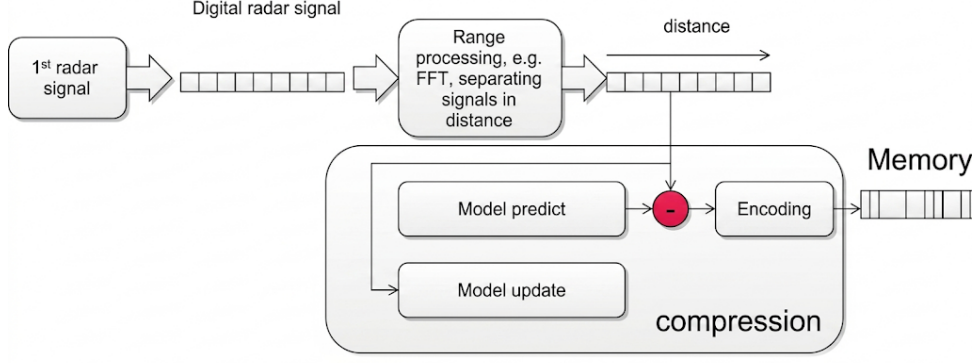


Figure 2.8: High-level pipeline of the Doppler Redundancy Hybrid Encoding (DRHE) algorithm [21]

AdaRadar first converts localized patches of radar data into the frequency domain using a Discrete Cosine Transform (DCT). Because radar reflections are highly concentrated at specific frequencies, the algorithm can aggressively discard the near-zero-frequency coefficients. The remaining essential data is then scaled and quantized.

The defining feature of AdaRadar is its closed-loop feedback mechanism. Rather than using a fixed compression rate, the algorithm continuously adjusts how aggressively it discards data based on the performance of a downstream perception task (such as an object detection neural network). This allows the sensor to heavily compress data and automatically adjust the compression ratio to preserve fine details.

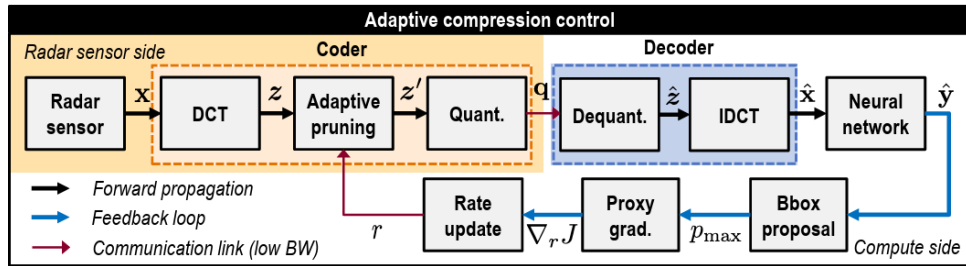


Figure 2.9: Conceptual pipeline and feedback loop of the AdaRadar spectral compression framework [23].

2.6 Field-Programmable Gate Arrays (FPGAs)

The implementation of edge-based radar data compression requires a hardware platform that bridges the gap between fixed-functionality Application-Specific Integrated Circuits (ASICs)

and flexible yet computationally slow general-purpose processors. Field-Programmable Gate Arrays (FPGAs) provide this essential middle ground, offering the performance of dedicated hardware with the flexibility of software-defined logic.

2.6.1 Strategic Advantages for Radar DSP

FPGAs are to be used for the hardware implementation due to several advantages over other computing devices.

Massive Parallelism and Deterministic Latency

The biggest advantage of FPGAs in automotive radar applications is their ability to do parallel processing. A microcontroller must process the radar data one after another, whilst an FPGA can create an array of parallel pipelines to process multiple chirps and virtual antenna channels simultaneously [27]. This allows for deterministic latency, meaning the time required to compress the data is low and predictable.

Hardware-Software Co-Design

Modern System-on-Chip (SoC) FPGAs, such as the AMD Zynq UltraScale+ family, integrate Programmable Logic (PL) with a dedicated Processing System (PS) based on ARM Cortex processors. This enables a co-design approach where control-plane logic and management tasks run in software, while data-plane tasks (such as non-linear quantization and DRHE compression) are offloaded to hardware accelerators in the PL.

Reconfigurability for Iterative Design

Given the experimental designs explored in this thesis, reconfigurability is vital. The ability to modify hardware logic using Hardware Description Languages (HDLs) such as VHDL or C++ via High-Level Synthesis (HLS) enables iteration and testing without the cost and lead times associated with ASICs.

2.6.2 Deployment and Production Considerations

While FPGAs are the gold standard for prototyping and low- to medium-volume production, their cost and power consumption can make it undesirable for actual automotive use. In a

commercial vehicle, the compression core developed on an FPGA is typically transitioned into one of two architectures for mass production:

- **ASIC Integration:** For high-volume models, the logic is permanently etched into a custom chip, yielding the lowest power consumption and highest performance.
- **SoC Hardware Accelerators:** The compression logic is integrated as a dedicated fixed-function block within a larger automotive SoC (similar to the NXP S32R series), providing a balance between cost-effectiveness and performance [20].

This research focuses on the FPGA implementation as a critical proof of concept.

2.7 Hardware Number Representations

In the context of implementing automotive radar compression algorithms on hardware (such as FPGAs or ASICs), the choice of number representation affects the latency, resource consumption, and signal fidelity. The two main methods of representing numbers are fixed-point and floating-point.

2.7.1 Fixed-Point Representation

Fixed-point representation uses a constant number of bits for the integer portion and for the fraction portion of the number. This is commonly denoted using Q-format (e.g., $Q_{m.n}$, where m bits represent the integer, n bits represent the fraction, and an optional bit is reserved for the sign).

The main benefit of using fixed-point arithmetic is that it's very hardware efficient [28]. Since the position of the decimal is fixed, the hardware does not need to have dynamic location functionality. Mathematical operations such as addition, subtraction, and multiplication can be performed using the standard Arithmetic Logic Unit (ALU). This allows for lower latency and reduced resource usage.

However, fixed-point representation has a restricted dynamic range. If a signal is larger than the m integer bits can represent, then an overflow occurs, which wraps the number around to a negative/incorrect value. Conversely, if a signal is smaller than the n fractional bits can resolve, an underflow would occur, which typically rounds to zero. Hence, we must ensure that scaling

is applied in the pipeline to keep the data within the dynamic range.

2.7.2 Floating-Point Representation

Floating-point representation dynamically adjusts the position of the decimal point by separating the number into three components: a sign bit, an exponent, and a mantissa [29]. The final value is calculated as $\text{Sign} \times \text{Mantissa} \times 2^{\text{Exponent}}$.

The advantage of using floating-point arithmetic is its dynamic range. Since the exponent acts as a sliding decimal point, we can maintain precision for small values whilst still representing larger values. This removes the need for bit-scaling.

Despite this flexibility, floating-point numbers have a massive hardware disadvantage. Performing basic arithmetic requires aligning the exponents before adding the mantissas, normalizing the result, and handling special cases. This necessitates a dedicated, highly complex hardware block known as a Floating-Point Unit (FPU) [28]. Using FPUs on hardware consumes a vast number of logic gates and increases power consumption. More critically for automotive radar, FPU operations generally require multiple clock cycles to complete, resulting in higher latency. Because edge compression algorithms must process gigabits of ADC data per second with strict timing, floating-point math is generally too slow and resource-intensive.

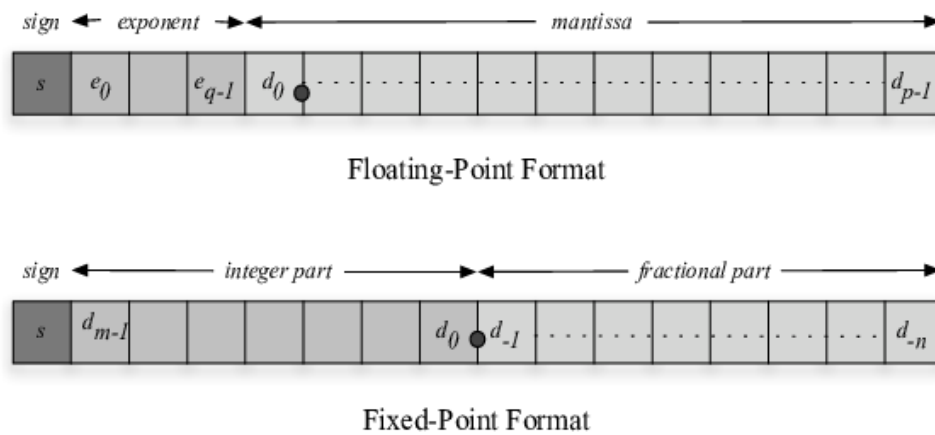


Figure 2.10: Fixed vs Float Representation [30]

2.7.3 Quantization Error in Fixed-Point Systems

Because edge hardware predominantly relies on fixed-point arithmetic to meet strict latency requirements, analog signals must eventually be mapped to discrete fixed-point values. The

discrepancy between the ideal continuous value and its nearest discrete approximation is known as quantization error [31].

When converting to a fixed-point grid, the dynamic range is divided by a uniform step size, Δ . If a signal value falls between two steps, it must be rounded or truncated. This rounding introduces an inherent error bounded by $\pm \frac{\Delta}{2}$. In signal processing, this error acts as a wideband noise floor, commonly referred to as quantization noise.

This error is highly problematic for automotive radar compression. If a linear fixed-point quantization scheme is applied, there will naturally be data that is lost

Consequently, the subtle variations of weak signals are crushed into the quantization noise floor and lost during compression. This limitation of linear fixed-point quantization is the primary motivation for exploring other means to linear quantization, ensuring that critical weak target data survives the compression.

Chapter 3

Thesis Outline

This chapter outlines the structure and trajectory of this thesis.

3.1 Baseline Study: Kiem’s Thesis

3.1.1 Overview and Objectives

The baseline for this thesis is the master’s thesis by Lukas Kiem [12], titled *Design and Implementation of Data Compression Methods for SoC-based Radar Systems*. His research was motivated by the same problem statement. His primary objective was to design, evaluate, and implement an automotive radar data compression algorithm on an AMD Zynq UltraScale+ MPSoC.

3.1.2 Results and Algorithm Evaluation

To determine the most viable compression algorithm, Kiem conducted a comprehensive design space exploration. He compared and simulated a large variety of standard compression algorithms, including Texas Instruments’ Exponential-Golomb Encoding (EGE) variants, Infineon’s common exponent formats (CMPRA and CMPRB), and predictive coding methods (such as DRHE).

Upon comparing these algorithms in their performance metrics, specifically Compression Ratio (CR), Signal-to-Noise Ratio (SNR), and target detection accuracy, Kiem identified Doppler Redundancy Hybrid Encoding (DRHE) as the “best”. While the commercial deterministic

formats (like EGE and CMPRB) successfully reduced data size, they raised the noise floor and led to high rates of false-positive and false-negative detections. As summarized in Figure 3.1, DRHE achieved the highest compression ratio while maintaining good data fidelity and the lowest impact on the noise floor.

Algorithm	CR	FN (%)	FP (%)	NF est. (dBFS)	SNR (dB)
FP16	1	0	0	-70.714	22.407
FX16	1	0.59	1.18	-70.691	22.33
CMPRA	2	0.59	0.59	-70.687	22.418
CMPRB	4	7.69	4.73	-70.253	22.111
EGE10	1.6	1.18	1.18	-70.689	22.403
EGE8	2	2.37	2.37	-70.64	22.289
EGE6	2.67	7.69	7.1	-70.367	22.197
EGE4	4	40.24	26.63	-74.689	28.245
DRHE	3.25	0.59	1.18	-70.691	22.33

Figure 3.1: Compression algorithm analysis done by Kiem [12].

3.1.3 Limitations and Future Work

Whilst Kiem did successfully demonstrate the viability of the DRHE algorithm, achieving a compression ratio of 3.2 with a latency of 0.08 milliseconds, his work left some gaps. His work had three main limitations:

Quantization Error from Fixed-Point Representation

The most significant issue lay in his implementation of the mandatory data conversion step before the actual compression. To meet the strict latency requirement, he implemented the algorithm using 16-bit fixed-point representation (FX16).

This use of linear fixed-point quantization introduced false positives and negatives, as seen in the results of both FX16 and DRHE in Figure 3.1. His thesis acknowledged this loss of precision as the source of degradation, noting that reducing quantization error was important for future implementations.

Omission of Newer State-of-the-Art Algorithms

Secondly, whilst his design space exploration was thorough, covering a wide range of compression algorithms, some remained uninvestigated, especially those that had recently emerged.

Incomplete End-to-End System Implementation

Lastly, the scope of his hardware implementation was limited to just the compression pipeline on the Zynq UltraScale+ MPSoC; he did not implement the full compression and decompression pipeline on hardware. To fully evaluate the algorithm's performance, the full pipeline needs to be implemented.

3.2 Proposed Thesis Plan

To address the identified limitations, this thesis proposes the following goals.

3.2.1 Goal 1: Mitigating Quantization Error

The first major goal of this research is to resolve the quantization noise caused by the FX16 conversion.

To resolve this, this thesis will investigate two primary solutions:

1. **Piecewise Transformation:** Inspired by the work seen in *Synthetic Aperture Radar Image Compression Based on Low-Frequency Rejection and Quality Map Guidance* [14], this approach would involve applying a piecewise transformation function to the data before it is quantized. By expanding the dynamic range allocated to weak signals, they can now survive quantization.
2. **Floating-Point Hardware Implementation:** Alternatively, this research will explore implementing the algorithm using floating-point arithmetic. Since the latency seen by Kiem was 0.08 milliseconds [12], it was suggested that there would be enough headroom to implement the algorithm using floating-point arithmetic.

3.2.2 Goal 2: Evaluation of Novel Compression Algorithms

The second goal broadens the scope of the study by evaluating contemporary state-of-the-art compression techniques.

Utilizing a simulation environment built in PyTorch and MATLAB, the optimized DRHE algorithm will be benchmarked against three compression paradigms:

- NXP CP4/CP8: Industry-standard deterministic block floating-point formats.
- Block Adaptive Quantization (BAQ): Statistically adaptive spatial compression.
- AdaRadar: Compression rate-adaptive spectral compression relying on DCT and perception feedback loops.

These algorithms will be compared based on their compression ratios, Signal-to-Noise Ratio (SNR) retention, and direct impact on downstream target-detection accuracy (measured by false-positive and false-negative rates).

3.2.3 Goal 3: End-to-End Hardware Simulation on the EVM

The last goal is to implement the full compression and decompression pipeline. The most effective algorithm identified in the previous goals will be translated over to FPGA hardware.

To validate the design in a realistic automotive architecture, the full setup will involve:

1. Data Capture (EVM): A Texas Instruments Evaluation Module (EVM) will be used as the radar, streaming high-throughput data.
2. Edge Compression (FPGA): The EVM will feed directly into an FPGA. The compression algorithm will encode the data in real time.
3. Central Processing (PC): The compressed bitstream will be transmitted to a connected PC. The PC, simulating the System-on-Chip (SoC), will execute the decompression algorithm.

This end-to-end setup will provide the quantitative evidence of the algorithms' performance in the real world

3.3 Project Timeline

To achieve the goals outlined above, they have been structured into a timeline that spans the duration of the thesis. The project progresses from the initial research into software simulation. Once the compression algorithms are modeled in software, the final part of the timeline is dedicated to the hardware implementation. A detailed breakdown of the timeline is illustrated in the Gantt chart in Figure 3.2.

Tasks	Days	Term 2 2026										Exams & Break			Term 3 2026										
		1	2	3	4	5	6	7	8	9	10	E	B1	B2	B3	1	2	3	4	5	6	7	8	9	10
Thesis B																									
Create "Baseline" Compression Model in MATLAB	14																								
Reduce Quantisation Error of DRHE	21																								
Implement Other Compression Models	28																								
Compare results of Compression Models	4																								
Start getting data from AWR EVM	31																								
Thesis C																									
FPGA Decompression Implementation	35																								
PC Modulation Implementation	14																								
Get Hardware results of full system (AWR -> FPGA -> PC)	7																								
Generate Results	7																								

Figure 3.2: Proposed project timeline and task breakdown.

3.4 Challenges and Risks

3.4.1 FPGA Implementation and Timing Closure

The biggest challenge in this project is implementing the compression algorithm on the FPGA. The hardware logic must meet the strict latency requirements. In hardware terms, this means achieving "timing closure" (ensuring that signals propagate through the FPGA logic gates within the required clock cycle), which requires meticulous optimization. Additionally, physical resource bottlenecks (such as the availability of DSP slices and Block RAM on the FPGA) pose a challenge when implementing mathematically dense algorithms.

3.4.2 Simulating Proprietary MCU Architectures

A unique challenge in Goal 2 is creating a fair comparative evaluation against existing commercial algorithms, particularly NXP's CP4 and CP8 formats. These compression algorithms are implemented as proprietary, hardware accelerators integrated into NXP's S32R MCUs. Simulating the exact behavior, latency, and resource efficiency of an MCU-specific hardware accelerator in a generic PyTorch/MATLAB software environment inherently introduces inaccuracies. Furthermore, attempting to synthesize a comparable version of these algorithms on a completely different FPGA architecture for benchmarking risks an imperfect "apples-to-oranges" hardware comparison. Navigating these architectural discrepancies to ensure a good evaluation will be a hurdle during the simulation phase.

Chapter 4

Conclusion

This report has detailed the bandwidth bottlenecks in modern automotive architectures caused by the escalating data volumes generated by cascaded MIMO radar systems. It has established the need for low-latency data compression at the edge sensor node. Building on the foundational DRHE algorithm evaluated in Kiem’s baseline study, this report identifies the limitations of linear fixed-point quantization. Consequently, a research plan was outlined to address these vulnerabilities. By proposing the implementation of non-linear piecewise transformations or floating-point arithmetic to mitigate quantization error, and by detailing a structured hardware evaluation of other compression algorithms on an EVM-FPGA setup, this thesis establishes a clear trajectory towards optimizing radar compression for automotive applications.

Bibliography

- [1] C. Waldschmidt, J. Hasch, and W. Menzel, “Automotive radar — from first efforts to future systems,” *IEEE Journal of Microwaves*, vol. 1, no. 1, pp. 135–148, 2021.
- [2] S. Yao *et al.*, “Exploring radar data representations in autonomous driving: A comprehensive review.” arXiv preprint, 2023.
- [3] M. I. Skolnik, *Radar Handbook*. New York, NY, USA: McGraw-Hill, 3rd ed., 2008.
- [4] Infineon Technologies AG, “Infineon’s new radar MMIC RASIC CTRX8191F enables the next generation of 4D and HD imaging radars.” Market News, 2024.
- [5] V. Bandur *et al.*, “Making the case for centralized automotive E/E architectures,” *IEEE Trans. Vehicular Technology*, vol. 70, no. 2, pp. 1230–1245, 2021.
- [6] Z. Liu, W. Zhang, and F. Zhao, “Impact, challenges and prospect of software-defined vehicles,” *Automotive Innovation*, vol. 5, no. 2, pp. 180–194, 2022.
- [7] T. Böhm, “RISC-V - success factors & opportunities for dependable automotive applications.” Presented at RISC-V European Summit, 2024.
- [8] Texas Instruments, “FPD-Link III serializer/deserializer (SerDes) for sensor applications.” Application Note SNLA344, 2021.
- [9] Maxim Integrated (Analog Devices), “GMSL SerDes technology for automotive applications.” White Paper, 2022.
- [10] K. Matheus and T. Königseder, *Automotive Ethernet*. Cambridge, UK: Cambridge University Press, 2021.

- [11] Semiconductor Engineering, “Automotive architectures.” https://semiengineering.com/knowledge_centers/automotive/automotive-architectures/, accessed 17/04/2026, 2024.
- [12] L. Kiem, “Design and implementation of data compression methods for SoC-based radar systems,” Master’s thesis, Inst. of Technical Informatics, Graz University of Technology, Graz, Austria, May 2025.
- [13] IEEE, “IEEE P802.3bp 1000BASE-T1 PHY task force.” <https://www.ieee802.org/3/bp/>, accessed 17/04/2026, 2016.
- [14] V. B. Polyakov, N. A. Ignatova, and A. A. Sentsov, “Synthetic aperture radar image compression based on low-frequency rejection and quality map guidance,” in *Proc. WECONF*, pp. 1–4, 2021.
- [15] O. Meteer and M. J. G. Bekooij, “Low-power sign-magnitude fft design for fmcw radar signal processing,” *Workshop on Design and Architectures for Signal and Image Processing (14th edition)*, pp. 52–59, 2021.
- [16] Texas Instruments, *AWR2243 Single-Chip 76- to 81-GHz FMCW Transceiver datasheet (Rev. D)*. Texas Instruments, February 2024.
- [17] K. Sayood, *Introduction to Data Compression*. Morgan Kaufmann, 5th ed., 2017.
- [18] Convex App, “What is data compression?” <http://www.convexapp.com/2019/06/06/what-is-data-compression/>, June 2019. Accessed: 2026-04-19.
- [19] A. Mani and Z. Yang, “Memory compression and decompression engine for TI mmwave radar,” tech. rep., Texas Instruments, 2019.
- [20] NXP Semiconductors, “S32R RADAR signal compression,” Application Note AN5375, NXP Semiconductors, 2021.
- [21] Z. Zivkovic and L. Li, “On-the-fly range-velocity radar data compression exploring Doppler spectrum redundancy,” 2015.
- [22] F. Cheng *et al.*, “Autoencoder-based feature compression for radar and vision sensor fusion in autonomous driving,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 8, pp. 12345–12355, 2021.

- [23] J. Park, S. Y. Chun, and M. Seok, “AdaRadar: Rate adaptive spectral compression for radar-based perception,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.
- [24] I. Barabás *et al.*, “Rethinking vehicle architecture through softwarization and servitization,” *IEEE Access*, vol. 12, pp. 110782–110795, 2024.
- [25] R. Kwok and W. Johnson, “Block adaptive quantization of magellan sar data,” *IEEE Transactions on Geoscience and Remote Sensing*, vol. 27, no. 4, pp. 375–383, 1989.
- [26] Y. Cheng, J.-y. Song, X.-y. Zhao, F. Xu, and Y.-x. Peng, “Design and implementation of raw data compression system for subsurface detection SAR based on FPGA,” *Journal of Signal Processing*, vol. 35, no. 11, pp. 1799–1807, 2019.
- [27] I. Kuon, R. Tessier, and J. Rose, “Fpga architecture: Survey and challenges,” *Foundations and Trends in Electronic Design Automation*, vol. 2, no. 2, pp. 135–253, 2008.
- [28] B. Parhami, *Computer Arithmetic: Algorithms and Hardware Designs*. Oxford University Press, 2nd ed., 2010.
- [29] “IEEE standard for floating-point arithmetic,” 2019.
- [30] X. Guan, Y. Zhang, J. Wu, and J. Zhao, “Enhancing the implementation of mathematical formulas for fixed-point and floating-point arithmetics,” in *Proceedings of the 2023 6th International Conference on Computer Science and Software Engineering (CSSE 2023)*, pp. 141–147, 2023.
- [31] B. Widrow and I. Kollár, *Quantization Noise: Roundoff Error in Digital Computation, Signal Processing, Control, and Communications*. Cambridge University Press, 2008.

Appendix - Risk Assessment

HS017

HS Risk management form

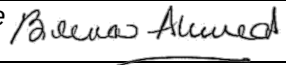


For additional information refer to HS329 [Risk Management Procedure](#)

Faculty/Division: Engineering		School/Unit: School of Electrical Engineering and Telecommunications			
Document number	HS_EET_001	Initial Issue date	22.04.2026	Current version	V1
		Current Version Issue date	22.04.2026	Next review date	
				22.04.2027	

Risk management name

Honours Thesis - Radar Data Compression Hardware Implementation

Form completed by	Jordan Mohindra	Signature	Jordan	Date	22/04/2026
Responsible supervisor/ authorising officer		Signature		Date	22/04/2026

Identify the activity and the location of the activity

Description of activity I am comparing and implementing data compression methods for SoC-based radar systems. This involves running software simulations (MATLAB/PyTorch), as well as hardware deployment involving programming an FPGA and interfacing it with the MMWCAS-RF-EVM

Description of location EE&T Laboratories and Personal Workstation at UNSW

Identify who may be at risk from the activity:

This may include fellow workers, visitors, contractors and the public. The types of people may affect the risk controls needed and the location may affect the number of people at risk

Persons at risk Myself (Jordan Mohindra) and other laboratory occupants

How they were consulted on the risk Standard Laboratory Safety Inductions

List legislation, standards, codes of practice, manufacturer's guidance etc used to determine control measures necessary

Work Health and Safety Act 2011
Work Health and Safety Regulation 2011
Texas Instruments AWR2243 Evaluation Module (EVM) Handling Instructions

Identify hazards and control the risks.

1. An activity may be divided into tasks. For each task identify the hazards and associated risks. Also list the possible scenarios which could sooner or later cause harm.
2. Determine controls necessary based on legislation, codes of practice, Australian standards, manufacturer's instructions etc.
3. List existing risk controls and any additional controls that need to be implemented
4. Rate the risk once all controls are in place using the matrix in HS329 Risk Management Procedure

SHADED GREY AREAS

If you need to determine whether it's reasonably practicable to implement a control, based on the risk complete the shaded grey columns

Feel free to resize the boxes to suit your situation/the amount of text you need to use

Task/ Scenario	Hazard	Associated harm	Existing controls	Any additional controls required?	Risk Rating			Cost of controls (in terms of time, effort, money)	Is this reasonably practicable Y/N
					C	L	R		
Using the MMWCAS-RF-EVM and FPGA Boards. Potential for electric shock from faulty power supplies or cabling	Electrical	Electric Shock, Burns	I am visually inspecting all cables and power supplies prior to use. All equipment is connected to lab circuits protected by Residual Current Devices (RCDs). I am using tested & tagged equipment	No	3	E	M	N/A	Y
Prolonged computer use for PyTorch/MATLAB simulation and writing HDL code. Poor posture or lack of breaks.	Ergonomic / Manual handling	Musculoskeletal strain, eye fatigue	I am using an adjustable ergonomic lab chair and ensuring my monitor is at eye level. I will take regular screen breaks (e.g., 5 minutes every hour of coding).	No	2	C	M	N/A	Y

RISK RATING METHODOLOGY AND MATRIX

Consider the Consequences	Consider the Likelihood	Calculate the Risk																																											
Consider: What type of harm could occur (minor, serious, death)? Is there anything that will influence the severity (e.g. proximity to hazard, person involved in task etc.). How many people are exposed to the hazard? Could one failure lead to other failures? Could a small event escalate?	Consider: How often is the task done? Has an accident happened before (here or at another workplace)? How long are people exposed? How effective are the control measures? Does the environment effect it (e.g. lighting/temperature/pace)? What are people's behaviours (e.g. stress, panic, deadlines) What people are exposed (e.g. disabled, young workers etc.)?	1.Take the consequences rating and select the correct column 2.Take the likelihood rating and select the correct row 3. Select the risk rating where the two ratings cross on the matrix below. VH = Very high, H = High, M = Medium, L = Low																																											
5. Severe: death or permanent disability to one or more persons 4. Major: hospital admission required 3. Moderate: medical treatment required 2. Minor: first aid required 1. Insignificant: injuries not requiring first aid	A. Almost certain: expected to occur in most circumstances B. Likely: will probably occur in most circumstances C. Possible: might occur occasionally D. Unlikely: could happen at some time E. Rare: may happen only in exceptional circumstances	<table><tr><th colspan="2" rowspan="2"></th><th colspan="5">CONSEQUENCES</th></tr><tr><th>1</th><th>2</th><th>3</th><th>4</th><th>5</th></tr><tr><th rowspan="5">LIKELIHOOD</th><th>A</th><td>M</td><td>H</td><td>H</td><td>VH</td><td>VH</td></tr><tr><th>B</th><td>M</td><td>M</td><td>H</td><td>H</td><td>VH</td></tr><tr><th>C</th><td>L</td><td>M</td><td>H</td><td>H</td><td>VH</td></tr><tr><th>D</th><td>L</td><td>L</td><td>M</td><td>M</td><td>H</td></tr><tr><th>E</th><td>L</td><td>L</td><td>M</td><td>M</td><td>M</td></tr></table>			CONSEQUENCES					1	2	3	4	5	LIKELIHOOD	A	M	H	H	VH	VH	B	M	M	H	H	VH	C	L	M	H	H	VH	D	L	L	M	M	H	E	L	L	M	M	M
		CONSEQUENCES																																											
		1	2	3	4	5																																							
LIKELIHOOD	A	M	H	H	VH	VH																																							
	B	M	M	H	H	VH																																							
	C	L	M	H	H	VH																																							
	D	L	L	M	M	H																																							
	E	L	L	M	M	M																																							

Risk level	Required action
Very high	Act immediately: The proposed task or process activity must not proceed. Steps must be taken to lower the risk level to as low as reasonably practicable using the hierarchy of risk controls
High	Act today: The proposed activity can only proceed, provided that: (i) the risk level has been reduced to as low as reasonably practicable using the hierarchy of risk controls and (ii) the risk controls must include those identified in legislation, Australian Standards, Codes of Practice etc. and (iii) the document has been reviewed and approved by the Supervisor and (iv) a Safe Working Procedure or Safe Work Method has been prepared and (v) the supervisor must review and document the effectiveness of the implemented risk controls
Medium	Act this week: The proposed task or process can proceed, provided that: (i) the risk level has been reduced to as low as reasonably practicable using the hierarchy of controls and (ii) the document has been reviewed and approved by the Supervisor and (iii) a Safe Working Procedure or Safe Work Method has been prepared.
Low	Act this month: Managed by local documented routine procedures which must include application of the hierarchy of controls.

List emergency procedures and controls**List emergency controls for how to deal with fires, spills or exposure to hazardous substances and/or emergency shutdown procedures**

In the event of an electrical fault or smoke from the FPGA/EVM, I will immediately isolate the power at the wall socket, ensure the area is safe, and notify the laboratory manager and my supervisor. For any medical emergencies (such as electric shock), I will contact the nearest trained first aid officer or UNSW Security immediately.

Implementation

Implementation			
Additional control measures needed:	Resources required	Responsible person	Date of implementation
None identified at this stage. All required controls are standard lab practices currently in place.			

REVIEW			
Scheduled review date:			
Are all control measures in place?			
Are controls eliminating or minimising the risk?			
Are there any new problems with the risk?			
Review by: (name)			
Review date:			

Acknowledgement of Understanding

All persons performing these tasks must sign that they have read and understood the risk management (as described in HS329 Risk Management Procedure).

Note: for activities which are low risk or include a large group of people (e.g. open days, BBQ's, student classes etc), only the persons undertaking the key activities need to sign below. For all others involved in such activities, the information can be covered by other methods including for example a safety briefing, induction, and/or safety information sheet (ensure the method of communicating this information is specified here)

Risk management name and version number:		I have read and understand this risk management form	
Name		Signature	Date